

E1

A 你还记得我吗~

本题必须使用 C 语言提交。

题目描述

给定整数序列 $a_1, a_2, \dots, a_n$ 和 $b_1, b_2, \dots, b_m$ ，严格递增的非负整数序列 $A_1, A_2, \dots, A_n$ 和 $B_1, B_2, \dots, B_m$ ，求解如下多项式：

$$\left(\sum_{i=1}^n a_i x^{A_i}\right) + \left(\sum_{i=1}^m b_i x^{B_i}\right)$$

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 10^5$)，含义同题目描述。

第二行 n 个整数 $a_1, a_2, \dots, a_n$ ($0 < |a_i| \leq 10^9$)，含义同题目描述。

第三行 n 个非负整数 $A_1, A_2, \dots, A_n$ ($0 \leq A_i \leq 10^9$)，含义同题目描述。

第四行一个正整数 m ($1 \leq m \leq 10^5$)，含义同题目描述。

第五行 n 个整数 $b_1, b_2, \dots, b_m$ ($0 < |b_i| \leq 10^9$)，含义同题目描述。

第六行 n 个非负整数 $B_1, B_2, \dots, B_m$ ($0 \leq B_i \leq 10^9$)，含义同题目描述。

保证 $A_i > A_{i-1}$ 对 $1 < i \leq n$ 成立， $B_i > B_{i-1}$ 对 $1 < i \leq m$ 成立， $A_i = B_j \Rightarrow a_i + b_j \neq 0$ 对 $1 \leq i \leq n, 1 \leq j \leq m$ 成立。

输出格式

对于每组数据，输出三行：

- 第一行一个正整数 k ，表示所得多项式在合并同类项后有 k 个系数非 0 项，并设所得多项式为：

$$\sum_{i=1}^k c_i x^{C_i}$$

其中 $C_i > C_{i-1}$ 对 $1 < i \leq k$ 成立。

- 第二行 k 个整数 $c_1, c_2, \dots, c_k$ ，含义同上式。
- 第三行 k 个非负整数 $C_1, C_2, \dots, C_k$ ，含义同上式。

输入样例

```
1
3
1 2 3
1 2 3
3
2 3 4
2 3 4
```

输出样例

```
4
1 4 6 4
1 2 3 4
```

Hint

宋老师：「这道题，大一每年都做，但通过率都不高。」

```
#include <stdio.h>
#define maxn 200086

int t, n, m, k;
int a[maxn], A[maxn], b[maxn], B[maxn], c[maxn], C[maxn];

int main() {
    scanf("%d", &t);
    while(t--) {
        scanf("%d", &n);
        for(int i = 1; i <= n; i++) scanf("%d", &a[i]);
        for(int i = 1; i <= n; i++) scanf("%d", &A[i]);
        scanf("%d", &m);
        for(int i = 1; i <= m; i++) scanf("%d", &b[i]);
```

```

for(int i = 1; i <= m; i++) scanf("%d", &B[i]);
int posa = 1, posb = 1;
k = 0;
while(posa <= n || posb <= m) {
    if(posa > n) c[++k] = b[posb], c[k] = B[posb], posb++;
    else if(posb > m) c[++k] = a[posa], c[k] = A[posa], posa++;
    else if(A[posa] == B[posb]) c[++k] = a[posa] + b[posb], c[k] =
A[posa], posa++, posb++;
    else if(A[posa] < B[posb]) c[++k] = a[posa], c[k] = A[posa], posa++;
    else c[++k] = b[posb], c[k] = B[posb], posb++;
}
printf("%d\n", k);
for(int i = 1; i <= k; i++) printf("%d ", c[i]);
puts("");
for(int i = 1; i <= k; i++) printf("%d ", c[i]);
puts("");
}
}

```

B 小水獭和签到题

题目描述



小水獭在数学课上学习到了勾股定理，即直角三角形的直角边平方和等于斜边的平方。

善于举一反三的小水獭想知道对于给定正整数的 z, k ，存在多少对正整数 (x, y) 使得 $x^k + y^k = z^k$ 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 100$)，表示数据组数。

对于每组数据，一行两个正整数 z, k ($1 \leq z, k \leq 10^6$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个非负整数，表示满足 $x^k + y^k = z^k$ 的正整数对 (x, y) 的个数。

输入样例

```

2
2 1
5 2

```

输出样例

```

1
2

```

```

#include<stdio.h>
#include<math.h>
int main() {
    int t;
    scanf("%d",&t);
    while(t--) {
        int k, z, index = 0;
        scanf("%d%d",&z,&k);
        if (k > 2)
            printf("0\n");
    }
}

```

```

        else if( k == 1)
            printf("%d\n", z-1);
        else {
            for(long long i = 1; i < z; i++) {
                double j = sqrt(1ll * z * z - i * i);
                if(j == (int)j) {
                    index++;
                }
            }
            printf("%d\n", index);
        }
    }
    return 0;
}

```

C 小水獭和多路归并

题目描述



小水獭正在学习归并排序，它觉得将两个序列进行归并太简单了，因此想要尝试将多个序列进行归并，即多路归并。

具体来说，给定 k 个长度为 n 的序列 $[a_{1,1}, a_{1,2}, \dots, a_{1,n}]$, $[a_{2,1}, a_{2,2}, \dots, a_{2,n}]$, $\dots$, $[a_{k,1}, a_{k,2}, \dots, a_{k,n}]$ ，满足每个序列均是严格递增的，且这 kn 个数互不相同。小水獭会重复进行如下操作直到所有序列均为空：

- 对于所有非空序列的首个元素，找到最小的那个元素，并将其从所在序列中删除。

小水獭会进行 kn 次操作，它想请你帮它写一个程序模拟这个过程，输出每一次操作删掉了哪个数。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行两个正整数 k, n ($2 \leq k \leq 10^5$, $1 \leq n \leq 10^5$, $2 \leq nk \leq 10^5$)，表示序列个数和序列长度。

接下来 k 行，第 i 行 n 个整数 $a_{i,1}, a_{i,2}, \dots, a_{i,n}$ ($-10^9 \leq a_{i,j} \leq 10^9$)，表示第 i 个序列的元素。

保证 $a_{i,j} > a_{i,j-1}$ 对 $1 \leq i \leq k, 1 < j \leq n$ 成立，且 $a_{i,j} = a_{x,y} \Leftrightarrow (i = x \wedge j = y)$ 。

输出格式

对于每组数据，输出一行 kn 个整数，第 i ($1 \leq i \leq kn$) 个整数表示第 i 次操作删除的数。

输入样例

```

1
2 2
-1 3
-2 4

```

输出样例

```

-2 -1 3 4

```

```

#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

int t, k, n;

int main() {
    scanf("%d", &t);

```

```

while(t--) {
    scanf("%d%d", &k, &n);
    vector<int> v(k * n);
    for(int i = 0; i < k * n; i++) scanf("%d", &v[i]);
    sort(v.begin(), v.end());
    for(auto x : v) printf("%d ", x);
    puts("");
}
}

```

D 小水獭和随机数

题目描述



小水獭正在学习排序，不过这回它所排序的数字来自一个随机数生成器，它的 C++ 代码如下：

```

#include <bits/stdc++.h>
#define maxn 20000086

using namespace std;

unsigned seed;

int Rand(){
    seed ^= seed << 13;
    seed ^= seed >> 7;
    seed ^= seed << 17;
    return seed & ((1 << 24) - 1);
}

int t, n, a[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%u", &n, &seed);
        for(int i = 1; i <= n; i++) a[i] = Rand();
        sort(a + 1, a + 1 + n);
        unsigned ans = 0;
        for(int i = 1; i <= n; i++) ans += a[i] ^ i;
        printf("%u\n", ans);
    }
}

```

可惜这份代码 TLE 了，你能帮它修改一下代码，得到一份在本题数据范围限制下等价但速度更快的代码吗？

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，一行两个正整数 $n, seed$ ($1 \leq n \leq 2 \times 10^7$, $1 \leq seed < 2^{32}$)，表示序列的长度和初始随机种子。

输出格式

对于每组数据，按照小水獭原本代码的逻辑，输出一行一个非负整数。

输入样例

```

2
3 5
5 7

```

输出样例

```

23518363
40839186

```

样例解释

样例中两组数据对应的初始序列分别为：[9085253, 83271, 14349837] 和 [9363911, 93636, 5545503, 14569699, 10466434]。

```

#include <bits/stdc++.h>
#define maxn 20000086

```

```

using namespace std;

unsigned seed;

int Rand() {
    seed ^= seed << 13;
    seed ^= seed >> 7;
    seed ^= seed << 17;
    return seed & ((1 << 24) - 1);
}

int t, n, a[maxn];
int num[maxn];

int main() {
    scanf("%d", &t);
    while(t--) {
        memset(num, 0, sizeof(num));
        scanf("%d%u", &n, &seed);
        for(int i = 1; i <= n; i++) a[i] = Rand(), num[a[i]]++;
        unsigned ans = 0;
        int cnt = 0;
        for(int i = 0; i <= (1 << 24) - 1; i++) {
            while(num[i]--) cnt++, ans += i ^ cnt;
        }
        printf("%u\n", ans);
    }
}

```

E 小水獭和组合数

题目描述



小水獭正在学习「离散数学 3」，它接触到了组合数的定义，对于非负整数 n 和 m 有：

$$\binom{n}{m} = \frac{n!}{m!(n-m)!}$$

其中 $n! = 1 \times 2 \times \dots \times n$ ，特别地， $0! = 1$ 。

由组合意义可得，当 $n \geq m$ 时组合数是一个正整数。

小水獭希望你帮他编写一个程序用以计算组合数，特别地，如果组合数的值大于 10^9 ，你只需要输出 -1 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^5$)，表示数据组数。

对于每组数据，一行两个非负整数 n, m ($0 \leq m \leq n \leq 10^9$)，表示需要计算的组合数的两个参数。

输出格式

对于每组数据，输出一行一个整数：

- 如果 $\binom{n}{m} \leq 10^9$ ，输出 $\binom{n}{m}$ 。
- 否则，输出 -1 。

输入样例

```
2
4 2
100 50
```

输出样例

```
6
-1
```

```
#include <bits/stdc++.h>

using namespace std;

int t, n, m;

int main() {
    scanf("%d", &t);
    while(t--) {
        scanf("%d%d", &n, &m);
        m = min(m, n - m);
        long long x = 1;
        for(int i = 1; i <= m && x <= 1e9; i++) x *= n - i + 1, x /= i;
        printf("%d\n", x <= 1e9 ? x : -1);
    }
}
```

F 莫卡的二叉树魔法

题目描述

小水獭正在学习二叉树，它希望世界上所有的树都是二叉树。

为了实现这个目的，小水獭请求莫卡教自己二叉树魔法，使自己可以把世界上所有的树变成二叉树。

为了检验小水獭的魔法天赋，莫卡给了小水獭一棵有 n 个节点的二叉树，且根节点的编号为 1。这棵二叉树任意一个节点要么是白色，要么是黑色。之后莫卡会对这棵二叉树释放 q 次魔法，每次莫卡会选择一个节点，将以这个节点为根的子树内所有节点的颜色反转，即黑色变成白色，白色变成黑色。在莫卡的 q 次魔法全部释放完成之后，小水獭需要回答每个节点的颜色。

但是这个问题对于小水獭来说过于困难了，请聪明的你帮他回答这个问题吧。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 10^5$)，表示二叉树的节点数量。

第二行 $n - 1$ 个正整数，第 i ($1 \leq i \leq n - 1$) 个数表示编号为 $i + 1$ 的节点的父亲节点编号，数据保证是一棵二叉树。

第三行一个长度为 n 的 01 串，从左到右第 i ($1 \leq i \leq n$) 位如果为 0，表示编号为 i 的节点颜色为白色，否则为黑色。

第四行一个正整数 q ($1 \leq q \leq 10^5$)，表示魔法的释放次数。

接下来 q 行每行一个正整数 a_i ($1 \leq a_i \leq n$)，表示第 i 次魔法选择的节点编号。

输入格式

对于每组数据，输出一行一个长度为 n 的 01 串，表示 q 次魔法全部释放完成之后每个节点的颜色。从左到右第 i ($1 \leq i \leq n$) 位如果为 0，表示编号为 i 的节点颜色为白色，否则为黑色。

输入样例

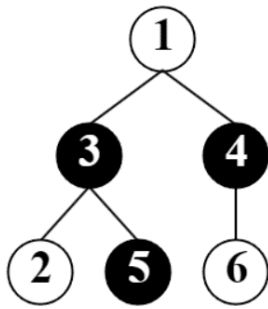
```
1
6
3 1 1 3 4
001110
3
1
3
2
```

输出样例

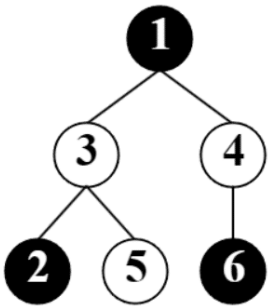
```
111011
```

样例解释

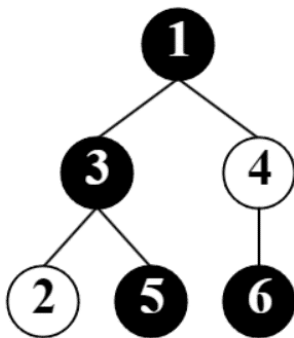
二叉树最初如下图所示：



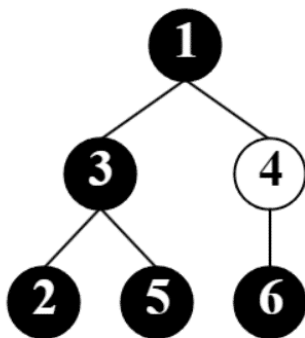
释放第 1 次魔法后，二叉树如下图所示：



释放第 2 次魔法后，二叉树如下图所示：



释放第 3 次魔法后，二叉树如下图所示：



因此最后的颜色序列为 111011。

```
#include<bits/stdc++.h>
#define MIN(a,b) a<b?a:b
using namespace std;
vector<int>a[100005];
int b[100005];
bool path[100005];
char s[100010];
```

```

void dfs(int u) {
    if(a[u].empty()) {
        return;
    }
    for(int i=0; i<a[u].size(); i++) {
        b[a[u][i]]^=b[u];
        dfs(a[u][i]);
    }
}

int main() {
    int n,q;
    int x;
    int T;
    scanf("%d", &T);
    while(T--) {
        scanf("%d",&n);
        for(int i = 1; i <= n; i++) a[i].clear(), b[i] = 0;
        for(int i=2; i<=n; i++) {
            scanf("%d",&x);
            a[x].push_back(i);
        }
        scanf("%s",s);
        scanf("%d",&q);
        while(q--) {
            scanf("%d",&x);
            b[x]^=1;
        }
        dfs(1);
        for(int i=0; i<n; i++) {
            printf("%d",(s[i]-'0')^b[i+1]);
        }
        puts("");
    }
}

```

G XIAO7 的猫粮

题目描述

成功变成猫猫的 XIAO7 开设了一家猫娘乐园，但由于猫猫太多，XIAO7 的猫粮不够吃了。

于是天上传来一个声音：「如果你能答对以下若干个问题，就会有足够多的猫粮从天而降。」

请你救救 XIAO7 和她的猫猫们！

具体来说，给定一个整数 n ，不妨将其看作一个十进制数字串，规定如下两种操作：

- 删除数字串中的任何一位数字（允许执行此操作而出现数字串为空的情况）。
- 在数字串中的最右侧添加一位数字。

以上操作可以按任意顺序执行任意次数。请问最少需要执行多少次操作才能使数字串对应的整数成为 2 的任意非负整数次幂（即 $2^i, i \in \mathbb{N}$ ）？

最终得到的数字串不能有前导零。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^4$)，表示数据组数。

对于每组数据，一行一个正整数 n ($1 \leq n \leq 10^9$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个非负整数，表示最少操作次数。

输入样例

```
8
1256
701
12
95
512
7635
25765
100000
```

输出样例

```
1
2
1
3
0
4
2
5
```

样例解释

正确答案对应的操作种类和顺序可能不唯一。

- $n = 1256$, 答案为 1, 删除数字 1。
- $n = 701$, 答案为 2, 删除数字 7 和 0, 得 $1 = 2^0$ 。
- $n = 12$, 答案为 1, 在右侧添加数字 8, 得 $128 = 2^7$ 。
- $n = 95$, 答案为 3, 删除数字 9 和 5, 随后添加数字 8, 得 $8 = 2^3$ 。
- $n = 512$, 答案为 0, $512 = 2^9$, 无需执行操作。
- $n = 7635$, 答案为 4, 删除数字 3 和 5, 右侧添加数字 4, 删除数字 7, 得 $64 = 2^6$ 。
- $n = 25765$, 答案为 2, 删除数字 7 和最后一位数字 5, 得 $256 = 2^8$ 。
- $n = 100000$, 答案为 5, 删除所有的数字 0, 得 $1 = 2^0$ 。

Hint

XIAO7: 发出喵喵的声音~

AUTHOR: YUKI

```
#include<iostream>
#include<cstdio>
#include<cmath>
using namespace std;
typedef long long ll;
const int N = 55;
const int inf = 0x3f3f3f3f;
int getint() {
    char c;
    bool flag=0;
    int num=0;
    while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
    while(c>='0'&&c<='9') {
        num=num*10+c-48;
        c=getchar();
    }
    return flag ? -num : num;
}
int A[N], B[N];
int match(int x, ll target) {
    int n = 0, m = 0, i, j;
    while(x)
        A[++ n] = x % 10, x /= 10;
    while(target)
        B[++ m] = target % 10, target /= 10;

    for(i = m, j = n; i && j; i --, j --) {
        for(; j && B[i] != A[j]; j --);
        if(j <= 0) break ;
    }
}
```

```

int delete_num = n - (m - i);
int add_num = i;

return delete_num + add_num;
}
int main() {
    int T = getint();
    while(T --) {
        int n = getint(), ans = inf;
        for(int i = 0; i <= 60; i ++){
            ans = min(ans, match(n, 1ll << i));
        }
        cout << ans << endl;
    }
    return 0;
}

```

H 任务达人莫卡 I

题目描述



「高价收购致命的短枪，卖的密！」

莫卡穿越到了一个新的世界并且化身成了名为任务达人的 NPC。

现在莫卡手里有 n 个任务，编号为 $1, 2, \dots, n$ ，难度分别为 $a_1, a_2, \dots, a_n$ 。对于每一名前来领取任务的旅行者，莫卡会从中任意选取 3 个不同的任务，设编号为 i, j, k ($i \neq j, j \neq k, k \neq i$)。但是每个旅行者对任务的难度有独特的要求，他们会提出两个参数 l 和 r ，要求 $2 \cdot a_i \leq a_j \leq l \cdot a_i$ 且 $2 \cdot a_j \leq a_k \leq r \cdot a_j$ 。

现在莫卡想知道，对于每位旅行者，满足其难度要求的任务选择方案有多少种？由于答案可能很大，你只需要输出答案对 $10^9 + 7$ 取模后的结果。

两个选择方案（设任务编号分别为 i, j, k 和 i', j', k' ）被视作不同的方案当且仅当 $i \neq i'$ 或 $j \neq j'$ 或 $k \neq k'$ 。

输入格式

第一行一个正整数 n ($3 \leq n \leq 10^5$)，表示任务的数量。

第二行 n 个正整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^3$)，表示每个任务的难度。

第三行一个正整数 q ($1 \leq q \leq 10^5$)，表示旅行者的数量。

接下来 q 行每行两个正整数 l_i, r_i ($2 \leq l_i, r_i \leq 10^3$)，表示第 i 个旅行者提出的两个参数。

输出格式

输出 q 行，第 i 行一个非负整数，表示满足第 i 个旅行者难度要求的方案数对 $10^9 + 7$ 取模后的结果。

输入样例

```

4
1 2 2 4
1
2 3

```

输出样例

```

2

```

```

#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

```

```

const int p = 1e9 + 7;

int n, q;
int a[maxn];

int main() {
    scanf("%d", &n);
    for(int i = 1; i <= n; i++) {
        int x;
        scanf("%d", &x);
        a[x]++;
    }
    for(int i = 1; i <= 1000; i++) a[i] += a[i - 1];
    scanf("%d", &q);
    while(q--) {
        int l, r;
        scanf("%d%d", &l, &r);
        int ans = 0;
        for(int i = 1; i <= 1000; i++) {
            int x = a[i] - a[i - 1];
            int L = (i + l - 1) / l, R = i / 2;
            if(L > R) continue;
            int sum1 = a[R] - (L ? a[L - 1] : 0);
            L = i * 2, R = i * r;
            if(L > 1000) continue;
            R = min(R, 1000);
            int sum2 = a[R] - a[L - 1];
            ans = (ans + 1ll * x * sum1 % p * sum2) % p;
        }
        printf("%d\n", ans);
    }
}

```

I XIAO7 的地图

题目描述

XIAO7 的猫娘乐园中有一条很长的几乎覆盖了整个园区的路径。

这天 Couteau 前来参观，XIAO7 却画不出地图，只能请你帮帮她 => <=。

猫娘乐园的地图是一种递归定义的分形图，下面给出大小为 1, 2, 3 的地图图示，相信聪明的你一定可以总结出规律来~

在下面的图示中，纯白色的格子表示空地，其他颜色的格子（灰色和各种蓝色）表示路径。

本题采用字符画的形式输出猫娘乐园的地图，其中路径用  表示，空地用  表示。

当猫娘乐园的大小为 1 时，对应的输出为：

```

***
*
***

```

当猫娘乐园的大小为 2 时，对应的输出为：

```

* *****
* * *
*** ***
*
*** ***
* * *
* *****

```

一行一个正整数 n ($1 \leq n \leq 10$)，表示猫娘乐园的大小。

按题目描述中的规定，输出对应大小猫娘乐园的字符画。

XIAO7: 发出呜喵的声音~



```
#include<iostream>
#include<cstdio>
using namespace std;
typedef long long ll;
const int N = 3005;
int getint() {
    char c;
    bool flag=0;
    int num=0;
    while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
    while(c>='0'&&c<='9') {
        num=num*10+c-48;
        c=getchar();
    }
    return flag ? -num : num;
}

bool mat[N][N], tep[N][N];
void setMat(int sx, int sy, int n) {
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            mat[sx + i][sy + j] = tep[1 + i][1 + j];
}

void dfs(int n, int m, int sx, int sy) {
```

```

if(n == 1) {
    mat[sx][sy] = mat[sx][sy + 1] = mat[sx][sy + 2] = 1;
    mat[sx + 1][sy] = mat[sx + 1][sy] = 0, mat[sx + 1][sy + 2] = 1;
    mat[sx + 2][sy] = mat[sx + 2][sy + 1] = mat[sx + 2][sy + 2] = 1;
    return ;
}

dfs(n - 1, m >> 1, sx, sy);

int sz = m >> 1;
for(int i = 1; i <= sz; i++)
    for(int j = 1; j <= sz; j++) {
        tep[i][j] = mat[sx + i - 1][sy + j - 1];
        mat[sx + i - 1][sy + j - 1] = 0;
    }

setMat(sx, sy + sz + 1, sz);
setMat(sx + sz + 1, sy + sz + 1, sz);

for(int i = 1; i <= m; i++)
    mat[sx + sz][i] = mat[i][sy + sz] = 0;
mat[sx][sy + sz] = mat[sx + m - 1][sy + sz] = mat[sx + sz][sy + sz + 1] = 1;

for(int i = 1; i <= sz/2; i++)
    for(int j = i; j < sz - i + 1; j++) {
        int r = tep[i][j];
        tep[i][j] = tep[sz - j + 1][i];
        tep[sz - j + 1][i] = tep[sz - i + 1][sz - j + 1];
        tep[sz - i + 1][sz - j + 1] = tep[j][sz - i + 1];
        tep[j][sz - i + 1] = r;
    }

setMat(sx, sy, sz);

for(int i = 1; i <= sz/2; i++)
    for(int j = 1; j <= sz; j++)
        swap(tep[i][j], tep[sz - i + 1][j]);

setMat(sx + sz + 1, sy, sz);
}

int main() {
    int n = getInt(), m = 3;
    for(int i = 2; i <= n; i++)
        m = m << 1 | 1;
    dfs(n, m, 1, 1);

    for(int i = 1; i <= m; i++) {
        int j = m;
        for(; mat[i][j] == 0; j--);
        for(int k = 1; k <= j; k++)
            putchar(mat[i][k] ? '*' : ' ');
        putchar(10);
    }
    return 0;
}

```

J 小水獭和战狼

题目描述



小水獭正在沉迷电影「War Wolf」，所以它的算法作业只能请求你帮忙完成了~

具体来说，给定两个整数序列 $a_1, a_2, \dots, a_n$ 和 $b_1, b_2, \dots, b_n$ ，定义整数序列 $i_1, i_2, \dots, i_m$ ($m \geq 1$) 是一个沃沃序列当且仅当其满足如下三个条件：

- $1 \leq i_1 < i_2 < \dots < i_m \leq n$
- $a_{i_1} < a_{i_2} < \dots < a_{i_m}$
- $b_{i_1} < b_{i_2} < \dots < b_{i_m}$

你需要对于 $j = 1, 2, \dots, n$ 求解有多少个沃沃序列满足 $i_m = j$ ，由于答案可能很大，你只需要输出答案对 2^{32} 取模后的结果。

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 10^5$)，含义同题目描述。

第二行 n 个整数 $a_1, a_2, \dots, a_n$ ($-10^9 \leq a_i \leq 10^9$)，含义同题目描述。

第三行 n 个整数 $b_1, b_2, \dots, b_n$ ($-10^9 \leq b_i \leq 10^9$)，含义同题目描述。

输出格式

对于每组数据，输出一行 n 个非负整数，第 j ($1 \leq j \leq n$) 个数表示满足 $i_m = j$ 的沃沃序列数量对 2^{32} 取模后的结果。

输入样例

```
2
3
1 2 3
2 1 3
3
1 2 3
3 2 1
```

输出样例

```
1 1 3
1 1 1
```

```
#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

unsigned c[maxn];

void add(int x, unsigned y) {
    for(; x < maxn; x += x & -x) c[x] += y;
}

unsigned query(int x) {
    unsigned sum = 0;
    for(; x; x -= x & -x) sum += c[x];
    return sum;
}

int t, n;
int a[maxn], b[maxn];
unsigned f[maxn];
```

```

void solve(int l, int r) {
    if(l == r) {
        f[l]++;
        return;
    }
    int mid = l + r >> 1;
    solve(l, mid);
    vector<pair<int, int> > v;
    for(int i = l; i <= r; i++) v.push_back({a[i], -i});
    sort(v.begin(), v.end());
    for(auto it : v) {
        int i = -it.second;
        if(i <= mid) add(b[i], f[i]);
        else f[i] += query(b[i] - 1);
    }
    for(auto it : v) {
        int i = -it.second;
        if(i <= mid) add(b[i], -f[i]);
    }
    solve(mid + 1, r);
}

int main() {
    scanf("%d", &t);
    while(t--) {
        scanf("%d", &n);
        for(int i = 1; i <= n; i++) scanf("%d", &a[i]);
        vector<int> v;
        for(int i = 1; i <= n; i++) scanf("%d", &b[i]), v.push_back(b[i]);
        sort(v.begin(), v.end());
        v.erase(unique(v.begin(), v.end()), v.end());
        for(int i = 1; i <= n; i++) b[i] = lower_bound(v.begin(), v.end(), b[i])
- v.begin() + 1;
        for(int i = 1; i <= n; i++) f[i] = 0;
        solve(1, n);
        for(int i = 1; i <= n; i++) printf("%u ", f[i]);
        puts("");
    }
}

```

E2

A 简单的可删堆

本题必须使用 C 语言提交。

题目描述

本题是堆的一种拓展应用，你的任务是实现一个小根堆，即根节点为所有元素最小值的堆，需要支持插入元素、查询堆顶元素、删除堆顶元素、删除堆中任意元素四个操作。

输入格式

第一行一个正整数 n ($1 \leq n \leq 10^5$)，表示操作的数量。

接下来 n 行，每行为一个操作，格式如下：

- 1 x : 向堆中插入元素 x ($1 \leq x \leq 10^9$)。
- 2: 删除堆顶元素。
- 3: 查询堆顶元素。
- 4 x : 删除堆中元素 x ($1 \leq x \leq 10^9$)，若有多个相同的元素，只需删除一个。

数据保证进行操作 2, 3 时堆非空，进行操作 4 时 x 必为堆中元素。

输出格式

对于每次查询堆顶元素时，输出一行一个正整数，表示此时堆顶元素的值。

在所有操作结束后，将堆中的元素从小到大依次输出到一行中。

输入样例

```
10
1 2
1 4
3
2
1 1
4 4
1 6
1 3
4 3
3
```

输出样例

```
2
1
1 6
```

Hint

堆的实现可以参考《算法导论》第三版的 84 页 - 93 页。

堆还有很多其他的实现方式，不同的堆有不同的性质，感兴趣的同学可以课后进行研究。

如果使用暴力删除的方式只可以获得部分分数，请思考复杂度正确的算法。

实在没有思路的同学可以考虑一下这道题的标题（所以究竟可删堆是什么呢？）。

```
#include<stdio.h>
#include<string.h>
#include<math.h>
#include<stdlib.h>
#include<ctype.h>
#define pi acos(-1)
#define inf 2147483647
#define llinf 9223372036854775807
#define MIN(a,b) (a<b?a:b)
#define MAX(a,b) (a>b?a:b)

#define N 200010
void swap(int *a,int *b) {
    int t=*a;
    *a=*b;
    *b=t;
}
int heap_size=0;
int heap[N];

int heapDel_size=0;
int heapDel[N];
```

```

void put(int x) {
    int now,next,res;
    heap[++heap_size]=x;
    now=heap_size;
    while(now>1) {
        next=now>>1;
        if(heap[now]>=heap[next]) {
            return;
        } else {
            swap(&heap[now],&heap[next]);
            now=next;
        }
    }
}

int get() {
    if(heap_size<1)exit(-1);
    return heap[1];
}

int del() {
    if(heap_size<1)exit(-1);
    int res=heap[1];
    heap[1]=heap[heap_size--];
    int now=1,next;
    while(now*2<=heap_size) {
        next=now*2;
        if(next<heap_size&&heap[next+1]<heap[next]) {
            next++;
        }
        if(heap[now]<=heap[next]) {
            return res;
        }
        swap(&heap[now],&heap[next]);
        now=next;
    }
    return res;
}

void putDel(int x) {
    int now,next,res;
    heapDel[++heapDel_size]=x;
    now=heapDel_size;
    while(now>1) {
        next=now>>1;
        if(heapDel[now]>=heapDel[next]) {
            return;
        } else {
            swap(&heapDel[now],&heapDel[next]);
            now=next;
        }
    }
}

int getDel() {
    if(heapDel_size<1)return -2;
    return heapDel[1];
}

```

```

int delDel() {
    int res=heapDel[1];
    heapDel[1]=heapDel[heapDel_size--];
    int now=1,next;
    while(now*2<=heapDel_size) {
        next=now*2;
        if(next<heapDel_size&&heapDel[next+1]<heapDel[next]) {
            next++;
        }
        if(heapDel[now]<=heapDel[next]) {
            return res;
        }
        swap(&heapDel[now],&heapDel[next]);
        now=next;
    }
    return res;
}

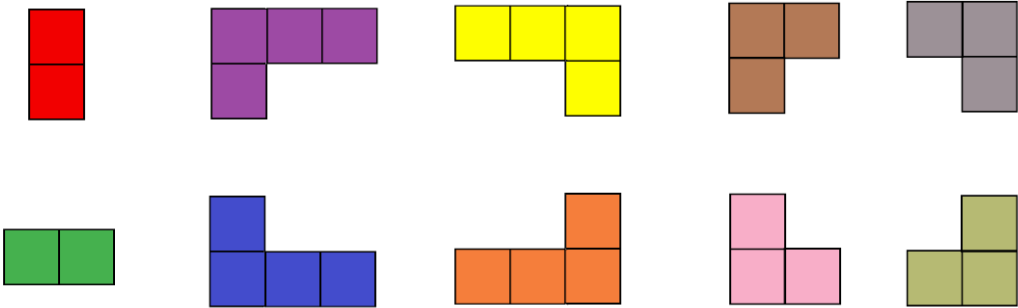
int main() {
    int n,op,x;
    scanf("%d",&n);
    while(n-->0) {
        scanf("%d",&op);
        if(op==1) {
            scanf("%d",&x);
            put(x);
        } else if(op==2) {
            while(get()==getDel()) {
                del();
                delDel();
            }
            del();
        } else if(op==3) {
            while(get()==getDel()) {
                del();
                delDel();
            }
            printf("%d\n",get());
        } else if(op==4) {
            scanf("%d",&x);
            putDel(x);
        }
    }
    while(heap_size>0) {
        if(get()==getDel()) {
            del();
            delDel();
        } else {
            printf("%d ",del());
        }
    }
    return 0;
}

```

B 比较简单的铺瓷砖

题目描述

有一个宽度为 2，长度为 n 的地板需要贴上瓷砖，有以下 10 种形状的瓷砖可供选择：



瓷砖不能旋转、翻转、重叠、切割，且必须恰好把地板铺满，可以结合输入输出样例理解。

the_ignorant 想知道一共有多少种不同的铺瓷砖方案，两种方案不同当且仅当存在一个位置上的瓷砖颜色不同。由于答案可能很大，你只需要输出答案对 998 244 353 取模后的结果。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^6$)，表示数据组数。

对于每组数据，一行一个正整数 n ($1 \leq n \leq 10^6$)，表示地板的长度。

输出格式

对于每组数据，输出一行一个非负整数，表示贴满瓷砖的方案数对 998 244 353 取模后的结果。

输入样例

```
5
1
2
3
4
5
```

输出样例

```
1
2
9
21
48
```

Hint

比较简单的递推，类似于斐波那契数。

P.S. 请枚举所有可能的情况，能过样例就证明你基本上对了。

P.P.S. 注意数据范围以及考虑递推与递归的区别。

```
#include<stdio.h>
#define ll long long int
#define M 1000100

ll MOD=998244353;
ll result[M]= {1, 1, 2, 9, 21, 48};
ll sum[M]= {1, 2, 4, 13, 34, 82};
int main() {
    int t, n, i, j;
    scanf("%d",&t);
    for(i=6; i<M; i++) {
        result[i] = (result[i-1] + result[i-2] + 4*result[i-3] + 2*result[i-4]
+2*sum[i-3])%MOD;
        sum[i] = (sum[i-1] + result[i])%MOD;
    }
    for(i=0; i<t; i++) {
        scanf("%d",&n);
```

```
printf("%11d\n",result[n]);  
}  
return 0;  
}
```

C 小水獭和子集

本题提交显示 Other Error 代表 Wrong Answer。

题目描述



小水獭在预推免面试过程中被问到了一个难题，想要紧急求助你，你能帮帮它嘛？

具体来说，给定正整数 n 和 m ，你需要构造 m 个非空正整数集合 $S_1, S_2, \dots, S_m$ ，满足：

- $S_i \subseteq \{1, 2, \dots, n\}$ 对 $1 \leq i \leq m$ 成立。
- $S_i \not\subseteq S_j$ 对 $1 \leq i, j \leq m, i \neq j$ 成立。

当然，题目有可能是无解的，此时你需要告知小水獭。

输入格式

第一行一个正整数 t ($1 \leq t \leq 2^{11}$)，表示数据组数。

对于每组数据，一行两个正整数 n, m ($1 \leq n \leq 10, 1 \leq m \leq 2^n - 1$)，含义同题目描述。

输出格式

对于每组数据，输出若干行：

- 若题目有解，输出 m 行，第 i ($1 \leq i \leq m$) 行第一个整数 $|S_i|$ ($1 \leq |S_i| \leq n$)，表示 S_i 的大小，接下来 $|S_i|$ 个正整数，表示 S_i 中的元素。如果有多种可能的答案，你可以输出任意一种。
- 否则，输出一行 -1 。

输入样例

```
4  
3 1  
3 2  
3 3  
3 4
```

输出样例

```
1 1  
1 1  
1 2  
1 1  
1 2  
1 3  
-1
```

```
#include <bits/stdc++.h>  
  
using namespace std;  
  
int t, n, m;  
  
int main(){  
    scanf("%d", &t);  
    while(t--){  
        scanf("%d%d", &n, &m);  
        int ans = 1;  
        for(int i = 1; i <= (n + 1) / 2; i++) ans *= n - i + 1, ans /= i;  
        if(ans < m){  
            puts("-1");  
        }
```

```

        continue;
    }
    for(int i = 0; i < (1 << n) && m; i++) if(__builtin_popcount(i) == (n + 1)
/ 2){
        m--;
        printf("%d ", (n + 1) / 2);
        for(int j = 0; j < n; j++) if(i & (1 << j)) printf("%d ", j +
1); puts("");
    }
}
}
}

```

D 莫卡和皇帝

题目描述

蛇皇帝扔给莫卡一个长度为 n 的序列 $a_1, a_2, \dots, a_n$, 初始有 $a_i = i$, 你需要维护一个数据结构, 支持 m 次下列三种操作之一:

1. 给定正整数 l, r , 对任意 $l \leq i \leq r$, 将 a_i 变为 a_i^2 .
2. 给定正整数 l, r, k , 对任意 $l \leq i \leq r$, 将 a_i 变为 $k \cdot a_i$.
3. 给定正整数 l, r , 输出 $\left(\prod_{i=l}^r a_i\right) \bmod (r-l+1)!$.

输入格式

第一行两个正整数 n, m ($1 \leq n, m \leq 10^6$), 含义同题目描述。

接下来 m 行, 每行代表一个操作, 为以下三种形式之一:

- 1 l r
- 2 l r k
- 3 l r

保证 $1 \leq l \leq r \leq n$ 且 $1 \leq k \leq 100$.

输出格式

对于每个第三种操作, 输出一行一个非负整数, 表示 $\left(\prod_{i=l}^r a_i\right) \bmod (r-l+1)!$ 的值。

输入样例

```

2 1
3 1 2

```

输出样例

```

0

```

```

#include <stdio.h>
int main(){
    int n, m, op, l, k, r;
    scanf("%d%d", &n, &m);
    while(m--){
        scanf("%d", &op);
        switch (op){
            case 1:
                scanf("%d%d", &l, &r);
                break;
            case 2:
                scanf("%d%d%d", &l, &r, &k);
                break;
            case 3:
                scanf("%d%d", &l, &r);
                printf("0\n");
        }
    }
}

```

```
    }  
    }  
    return 0;  
}
```

E 奇怪的汉诺獭

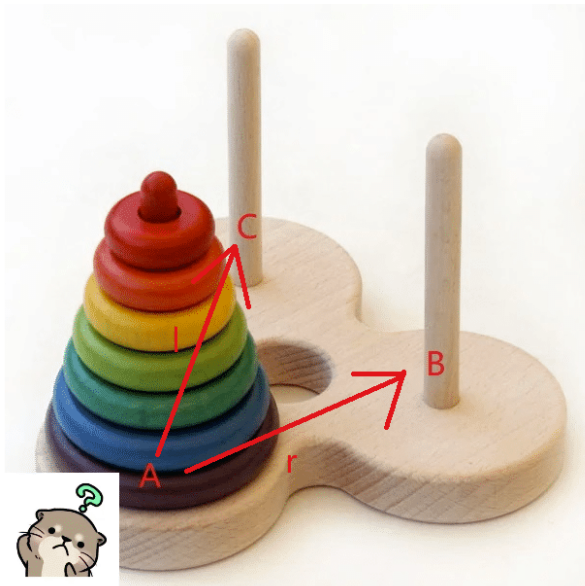
题目描述

小水獭学算法学累了，于是开始玩汉诺塔。它了解到，汉诺塔是一个有名的益智游戏，一共有三根柱子，其中一根柱子上从下往上按照大小顺序摆着 n 片圆盘，要求圆盘从下面开始按大小顺序重新摆放在另一根柱子上。并且规定，在小圆盘上不能放大圆盘，在三根柱子之间一次只能移动一个圆盘。

小水獭非常聪明，它很快明白了普通汉诺塔玩法，然而它还没有玩够，于是它构想了一个奇怪的汉诺塔。如下图所示，三柱均匀分布在一个圆形基座上，规定初始柱为 A 柱，目标柱为 C 柱，中间柱为 B 柱，且每个圆盘仅能依次顺时针旋转或逆时针旋转，其他规则和普通的汉诺塔相同。

举个例子，若圆盘 2 规定为仅能顺时针旋转，那么它仅能以 $A \rightarrow C \rightarrow B \rightarrow A$ 的顺序移动，若此时圆盘 2 在 A 柱，而圆盘 1 在 C 柱，则圆盘 2 无法移动，只能先设法移开圆盘 1 才能移动。

小水獭想问你如何以最少的操作次数，使所有圆盘从 A 柱移动到 C 柱。



输入格式

本题有多组数据，总组数不超过 10，请使用 EOF 判断读入结束。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 20$) 和一个长度为 n 的字符串 s ，其中 n 表示圆盘的数量， s 由 1 和 r 组成，第 i ($1 \leq i \leq n$) 代表第 i 小的圆盘所能旋转的方向， 1 代表顺时针方向 ($A \rightarrow C \rightarrow B \rightarrow A$)， r 代表逆时针旋转 ($A \rightarrow B \rightarrow C \rightarrow A$)。

输出格式

对于每组数据，输出一个操作方案。每个方案包含若干行操作，每步操作以形如 `move i from A to C` (意思是将第 i 小的圆盘从 A 柱移动到 C 柱) 的形式表现，在全部操作结束时，输出一个空行 (最后一组数据后有无空行不影响评测结果)。

输入样例

```
1 1  
1 r  
2 11
```

输出样例

```
move 1 from A to C  
  
move 1 from A to B  
move 1 from B to C  
  
move 1 from A to C  
move 1 from C to B  
move 2 from A to C  
move 1 from B to A  
move 1 from A to C
```

Hint

真是一个奇怪的汉诺獭，小水獭也表示很无奈。

```
#include <bits/stdc++.h>  
  
#define inf 1e18  
  
#define maxn 20005
```

```

typedef long long ll;
typedef unsigned long long ull;
using namespace std;
const ll mod = 998244353;
const double eps = 1e-4;
inline ll read(){
    ll x = 0, f = 1;char ch = getchar();
    while(ch > '9' || ch < '0'){if(ch == '-') f = -1;ch = getchar();}
    while(ch >= '0' && ch <= '9'){x = x * 10 + ch - '0';ch = getchar();}
    return x * f;
}

int n;
char s[maxn];

char pan[5] = {' ', 'A', 'B', 'C'};

void solve(int n, int now, int to){
    if(n == 1){
        if(to == now % 3 + 1) {
            if(s[n] == 'r') printf("move 1 from %c to %c\n", pan[now], pan[to]);
            else printf("move 1 from %c to %c\n", pan[now], pan[6 - now - to]),
printf("move 1 from %c to %c\n", pan[6 - now - to], pan[to]);
        }
        else{
            if(s[n] == 'l') printf("move 1 from %c to %c\n", pan[now], pan[to]);
            else printf("move 1 from %c to %c\n", pan[now], pan[6 - now - to]),
printf("move 1 from %c to %c\n", pan[6 - now - to], pan[to]);
        }
        return ;
    }
    if(to == now % 3 + 1) {
        if(s[n] == 'r') solve(n - 1, now, 6 - now - to), printf("move %d from %c
to %c\n", n, pan[now], pan[to]), solve(n - 1, 6 - now - to, to);
        else {
            solve(n - 1, now, to);
            printf("move %d from %c to %c\n", n, pan[now], pan[6 - now - to]);
            solve(n - 1, to, now);
            printf("move %d from %c to %c\n", n, pan[6 - now - to], pan[to]);
            solve(n - 1, now, to);
        }
    }
    else{
        if(s[n] == 'l') solve(n - 1, now, 6 - now - to),printf("move %d from %c
to %c\n", n, pan[now], pan[to]), solve(n - 1, 6 - now - to, to);
        else {
            solve(n - 1, now, to);
            printf("move %d from %c to %c\n", n, pan[now], pan[6 - now - to]);
            solve(n - 1, to, now);
            printf("move %d from %c to %c\n", n, pan[6 - now - to], pan[to]);
            solve(n - 1, now, to);
        }
    }
}

```

```
int main(){

    while(scanf("%d%s", &n, s + 1) != EOF){
        solve(n, 1, 3);
        puts("");
    }
    return 0;
}
```

F 小水獭和子数组

题目描述



小水獭在夏令营机试过程中遇到了一个难题，想要紧急求助你，你能帮帮它嘛？

具体来说，给定一个长度为 n 的数组 $a_1, a_2, \dots, a_n$ ，你需要求解其中所有子数组的极差的和。

形式化地说，定义 $S(l, r)$ ($1 \leq l \leq r \leq n$) 表示 $a_l, a_{l+1}, \dots, a_r$ 这些元素所组成的集合， $\max S$ 表示集合 S 中的最大值， $\min S$ 表示集合 S 中的最小值。请你求解：

$$\sum_{i=1}^n \sum_{j=i}^n [\max S(i, j) - \min S(i, j)]$$

输入格式

第一行一个正整数 t ($1 \leq t \leq 5$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 10^6$)，表示数组的长度。

第二行 n 个整数 $a_1, a_2, \dots, a_n$ ($-10^9 \leq a_i \leq 10^9$)，表示数组的元素。

输出格式

对于每组数据，输出一行一个整数，表示所有子数组的极差的和。

输入样例

```
2
5
1 2 3 4 5
5
233 233 233 233 233
```

输出样例

```
20
0
```

Hint

你可能需要使用 `__int128` 类型来存储最终的结果，但该类型无法使用常规的方式输出，这里给出一个该类型的输出实现：

```
void print(__int128 x){
    if(x < 10){
        printf("%d", (int)x);
        return;
    }
    print(x / 10);
    printf("%d", (int)(x % 10));
}
```

```
#include <bits/stdc++.h>
#define maxn 1000086
```

```
using namespace std;
```

```
int t, n;
int a[maxn];
int sk[maxn], top;
```

```

int l[maxn], r[maxn];

void print(__int128 x){
    if(x < 10){
        printf("%d", (int)x);
        return;
    }
    print(x / 10);
    printf("%d", (int)(x % 10));
}

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d", &n);
        for(int i = 1; i <= n; i++) scanf("%d", &a[i]);
        __int128 ans = 0;
        top = 0;
        for(int i = 1; i <= n; i++){
            while(top && a[sk[top]] <= a[i]) top--;
            if(top) l[i] = sk[top] + 1;
            else l[i] = 1;
            sk[++top] = i;
        }
        top = 0;
        for(int i = n; i; i--){
            while(top && a[sk[top]] < a[i]) top--;
            if(top) r[i] = sk[top] - 1;
            else r[i] = n;
            sk[++top] = i;
        }
        for(int i = 1; i <= n; i++) ans += (__int128)a[i] * (i - l[i] + 1) * (r[i]
- i + 1);
        top = 0;
        for(int i = 1; i <= n; i++){
            while(top && a[sk[top]] >= a[i]) top--;
            if(top) l[i] = sk[top] + 1;
            else l[i] = 1;
            sk[++top] = i;
        }
        top = 0;
        for(int i = n; i; i--){
            while(top && a[sk[top]] > a[i]) top--;
            if(top) r[i] = sk[top] - 1;
            else r[i] = n;
            sk[++top] = i;
        }
        for(int i = 1; i <= n; i++) ans -= (__int128)a[i] * (i - l[i] + 1) * (r[i]
- i + 1);
        print(ans); puts("");
    }
}

```

题目描述

czz 在离散数学课上对一只猫娘一见钟情，于是他找到猫娘乐园的园长 XIAO7 寻求帮助。

XIAO7 告诉他：「如果你能答对以下问题，下周三我就帮你略施小计。」

具体来说，在离散数学课上 czz 已经学到了组合数的定义，对于非负整数 n 和 m 有：

$$\binom{n}{m} = \frac{n!}{m!(n-m)!}$$

其中 $n! = 1 \times 2 \times \cdots \times n$ ，特别地， $0! = 1$ 。

由组合意义可得，当 $n \geq m$ 时组合数是一个正整数。

现在对于给定的三个整数 n, m, k ，XIAO7 想要知道 $\binom{n}{m}$ 在 k 进制下末尾 0 的数量。

输入格式

第一行一个正整数 t ($1 \leq t \leq 100$)，表示数据组数。

对于每组数据，每行三个整数 n, m, k ($0 \leq m \leq n \leq 10^9, 2 \leq k \leq 10^9$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个非负整数，表示 $\binom{n}{m}$ 在 k 进制下末尾 0 的数量。

输入样例

```
3
10 6 10
8 1 2
27 3 3
```

输出样例

```
1
3
2
```

样例解释

十进制下， $\binom{10}{6} = 210$ ，末尾有一个 0。

十进制下， $\binom{8}{1} = 8$ ，8 在二进制下为 1000，末尾有三个 0。

十进制下， $\binom{27}{3} = 2925$ ，2925 在三进制下为 11000100，末尾有两个 0。

Hint

XIAO7：发出喵喵的声音~



AUTHOR：YUKI

```
#include<iostream>
#include<cstdio>
#include<cmath>
#include<cstdlib>
#include<algorithm>
#include<vector>
#include<cstring>
#include<set>
#include<map>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
const int N = 3000005;
const int inf = 0x3f3f3f3f;
int getint(){
    char c;bool flag=0;int num=0;
```

```

while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
while(c>='0' && c<='9'){num=num*10+c-48; c=getchar();}
return flag ? -num : num;
}
int pri[N], num[N];
int getzero(int n, int x){
    int cnt = 0;
    while(n)
        cnt += n / x, n /= x;
    return cnt;
}
int getpri(int n){
    int h = sqrt(n + 0.5), cnt = 0;
    for(int i = 2; i <= h; i++){
        if(n % i) continue;
        pri[++cnt] = i, num[cnt] = 0;
        while(n % i == 0)
            n /= i, num[cnt]++;
    }
    if(n > 1) pri[++cnt] = n, num[cnt] = 1;
    return cnt;
}
int main(){
    int T = getint();
    while(T--){
        int n = getint(), m = getint(), k = getint();
        int cnt = getpri(k), ans = inf;
        for(int i = 1; i <= cnt; i++){
            int zero = getzero(n, pri[i]);
            zero -= getzero(m, pri[i]);
            zero -= getzero(n - m, pri[i]);
            ans = min(ans, zero / num[i]);
        }
        cout << ans << endl;
    }
}

```

H XIAO7 与 A+B problem

题目描述

猫娘「雪风」上周没有穿到裙子，她非常难过。

XIAO7 决定再给她一次机会：「如果你能答对以下若干个问题，我就给你买裙子。」

请你帮帮可爱的「雪风」！

具体来说，给定两个整数 A, B ，你需要找到一个整数 X 且 $|X| \leq 2^{64}$ 使得

$$(A \oplus X) + (B \oplus X) = X$$

其中 $\oplus$ 表示按位异或运算，所有数按照 128 位补码进行运算。

若不存在这样的 X 则请大声喊出 `I would like to see xf wear a skirt every day !`

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^5$)，表示数据组数。

对于每组数据，一行两个整数 A, B ($-2^{60} \leq A, B \leq 2^{60}$)。

输出格式

对于每组数据，输出一行：

- 若存在合法的 X ，输出 X ，容易证明此时仅有唯一的 X 满足条件。
- 否则，输出 `I would like to see xf wear a skirt every day !`。

输入样例

```
5
-1 -1
1 2
3 5
6 6
15 16
```

输出样例

```
I would like to see xf wear a skirt every day !
3
-8
4
31
```

Hint

XIAO7: 发出喵喵的声音~



AUTHOR: YUKI

```
#include<cstdio>
#include<cmath>
#include<cstring>
#include<cstdlib>
#include<iostream>
#include<algorithm>
#include<queue>
#include<set>
#include<map>
#include<vector>
typedef long long int LL;
typedef unsigned long long ULL;
const int INF=0x3f3f3f3f;
const LL INFLL=0x3f3f3f3f3f3f3f3fll;
LL getint(){
    LL ret=0;bool f=0;char c;
    while((c=getchar())<'0' || c>'9')if(c=='-')f^=1;
    while(c>='0'&&c<='9'){ret=ret*10+c-'0';c=getchar();}
    return f?-ret:ret;
}
using namespace std;
const int MAXN=100;
bool xa[MAXN+10],xb[MAXN+10],ans[MAXN+10];
void update(LL a,bool x[MAXN+10]){
    if(a>=0){
        memset(x,0,(MAXN+10)*(sizeof(bool)));
        for(int i=0;a;++i,a>=1)
            x[i]=a&1;
    }
    else{
        memset(x,1,(MAXN+10)*(sizeof(bool)));
        for(int i=0;~a;++i,a>=1)
            x[i]=a&1;
    }
}
int main(){
```

```

int T=getint();
while(T--){
    LL a=getint(),b=getint();
    if(a<0&&b<0){
        printf("I would like to see xf wear a skirt every day !\n");
        continue;
    }
    memset(ans,0,sizeof ans);
    update(a,xa);update(b,xb);
    bool c=0;
    for(int i=0;i<70;++i){
        ans[i]=c^xa[i]^xb[i];
        c=((ans[i]^xa[i])+(ans[i]^xb[i])+c)>1;
    }
    LL ret;
    if(ans[65]){
        ret=-1;
        for(int i=0;i<70;++i)if(!ans[i])
            ret-=(1ll<<i);
    }
    else{
        ret=0;
        for(int i=0;i<70;++i)if(ans[i])
            ret+=(1ll<<i);
    }
    cout<<ret<<'\n';
}
return 0;
}

```

I 小水獭和随机树（困难版）

题目描述



小水獭正在观摩一棵 n 个节点的树，其中每个点的权值来自一个随机数生成器，树的结构也在代码中直接钦定。它想知道对于每个点来说，所有与其距离小于 k 的祖先中（包括其本身），权值最大点的权值是多少？它的 C++ 代码如下：

```

#include <bits/stdc++.h>
#define maxn 10000086

using namespace std;

unsigned seed;

unsigned Rand(){
    seed ^= seed << 13;
    seed ^= seed >> 7;
    seed ^= seed << 17;
    return seed;
}

int t, n, k;
int fa[maxn];
unsigned a[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d%u", &n, &k, &seed);
        for(int i = 4; i <= n - 1000; i++) fa[i] = i / 2;
        for(int i = n - 998; i <= n; i++) fa[i] = i - 1;
        fa[n - 999] = 1;
        fa[2] = fa[3] = n;
        for(int i = 1; i <= n; i++) a[i] = Rand();
        unsigned ans = 0;
        for(int i = 1; i <= n; i++){
            unsigned mx = 0;
            int x = i;
            for(int j = 1; j <= k; j++){
                mx = max(mx, a[x]);
                if(x == 1) break;
                x = fa[x];
            }
            ans += mx ^ i;
        }
        printf("%u\n", ans);
    }
}

```

其中 `fa[i]` 表示编号为 `i` 的节点的父节点编号。

可惜这份代码 TLE 了，你能帮它修改一下代码，得到一份在本题数据范围限制下等价但速度更快的代码吗？

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，一行三个整数 $n, k, seed$ ($10^4 \leq n \leq 10^7$, $1 \leq k \leq n$, $1 \leq seed < 2^{32}$)，表示树的节点数、距离参数和初始随机种子。

输出格式

对于每组数据，按照小水獭原本代码的逻辑，输出一行一个非负整数。

输入样例

```

2
10000 3 5
100000 5 7

```

输出样例

```

2472146199
2180026278

```

Hint

树的形态是一条链拼上一棵二叉树。

```

#include <bits/stdc++.h>
#define maxn 10000086

using namespace std;

unsigned seed;

unsigned Rand(){
    seed ^= seed << 13;
    seed ^= seed >> 7;
    seed ^= seed << 17;
    return seed;
}

int t, n, k;
unsigned a[maxn];

```

```

unsigned ans;
int dep[maxn];
int q[maxn], l, r;

void dfs(int i){
    if(i > n) return;
    int L = l, R = r;
    while(l <= r && a[q[r]] <= a[i]) r--;
    int x = q[++r];
    q[r] = i;
    if(dep[i] - dep[q[l]] == k) l++;
    ans += a[q[l]] ^ i;
    if(i == 1){
        dep[n - 999] = dep[i] + 1;
        dfs(n - 999);
    }else if(i == n){
        dep[2] = dep[i] + 1, dfs(2);
        dep[3] = dep[i] + 1, dfs(3);
    }else if(i > n - 1000){
        dep[i + 1] = dep[i] + 1;
        dfs(i + 1);
    }else{
        if((i << 1) <= n - 1000){
            dep[i << 1] = dep[i] + 1;
            dfs(i << 1);
        }
        if((i << 1 | 1) <= n - 1000){
            dep[i << 1 | 1] = dep[i] + 1;
            dfs(i << 1 | 1);
        }
    }
    q[r] = x;
    l = L, r = R;
}

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d%u", &n, &k, &seed);
        for(int i = 1; i <= n; i++) a[i] = Rand();
        ans = 0, l = 1, r = 0, dfs(1);
        printf("%u\n", ans);
    }
}

```

J XIAO7 和序列

题目描述

猫娘 Asahi 每周都要前往沙河，她觉得好麻烦~！(´・ω・´)

XIAO7 告诉她：「如果你能答对以下若干个问题，我就能把你传送到沙河！」

请你帮帮可爱的 Asahi！

具体来说，给定一个长度为 n 的序列 $a_1, a_2, \cdots, a_n$ 。对于一次询问，给定两个正整数 $1 \leq l \leq r \leq n$ ，你需要求解 $a_l, a_{l+1}, \dots, a_r$ 中每种数字最靠右出现位置减去最靠左出现位置的和。

形式化地说，你需要求解：

$$\sum_{c \in S(l,r)} [R(l,r,c) - L(l,r,c)]$$

其中 $S(l,r)$ ($1 \leq l \leq r \leq n$) 表示 $a_l, a_{l+1}, \dots, a_r$ 这些元素所组成的集合， $L(l,r,c) = \min\{x \mid l \leq x \leq r, a_x = c\}$, $R(l,r,c) = \max\{x \mid l \leq x \leq r, a_x = c\}$ 。

你需要处理 q 次询问。

输入格式

第一行两个整数 n, q ($1 \leq n, q \leq 3 \times 10^5$)，表示序列的长度和询问次数。

第二行 n 个整数 $a_1, a_2, \cdots, a_n$ ($1 \leq a_i \leq 10^9$)，表示序列的元素。

接下来 q 行，第 i ($1 \leq i \leq q$) 行两个正整数 l_i, r_i ($1 \leq l_i \leq r_i \leq n$)，表示第 i 次询问的两个参数。

输出格式

输出 q 行，第 i ($1 \leq i \leq q$) 行一个非负整数，表示第 i 次询问的答案。

输入样例

```
7 5
1 2 2 3 1 2 3
1 3
1 4
2 6
2 7
2 5
```

输出样例

```
1
1
4
7
1
```

Hint

XIAO7：发出喵喵的声音~

AUTHOR：YUKI

```
#include<bits/stdc++.h>
#define maxn 3000005
using ll = long long;
using namespace std;

inline int read(){
    int x = 0, f = 1;char ch = getchar();
    while(ch > '9' || ch < '0'){if(ch == '-') f = -1;ch = getchar();}
    while(ch >= '0' && ch <= '9'){x = x * 10 + ch - '0';ch = getchar();}
    return x * f;
}

map<int, int> mp;

int a[maxn], n, q;
ll ans[maxn], bit[maxn];
void add(int x, int v){for(int i = x; i <= n; i += i & -i) bit[i] += v;}
ll getb(int x){ll ans = 0;for(int i = x; i; i -= i & -i) ans += bit[i];return ans;}
vector<pair<int, int>> v[maxn];

int main(){
```

```

n = read(), q = read();
for(int i = 1; i <= n; i++) a[i] = read();
for(int i = 1; i <= q; i++){
    int l = read(), r = read();
    assert(1 <= l && l <= r && r <= n);
    v[r].push_back({l, i});
}
for(int i = 1; i <= n; i++){
    if(mp.count(a[i])){
        int x = mp[a[i]];
        add(x, i - x);
    }
    mp[a[i]] = i;
    for(auto x: v[i]){
        ans[x.second] = getb(i) - getb(x.first - 1);
    }
}
for(int i = 1; i <= q; i++) printf("%11d\n", ans[i]);
}

```